

Part 1 Review

Project	Inventory Management System for B2B SaaS
Section	Part 1 - Code Review and Debugging
Focus	Reliability, validation, data integrity, and business rules

Quick take

The original endpoint works as a prototype, but it is too fragile for production. The biggest concerns are missing validation, duplicate SKU handling, and the use of two separate commits for one logical operation. That combination can create bad data even when the API appears to succeed.

Key issues and production impact

- **No request validation:** The code directly accesses fields like `data['name']` and `data['sku']` without checking if they exist. If any field is missing, it throws a `KeyError` and the request fails with a 500 error. If the request body isn't valid JSON, `request.json` becomes `None`, which leads to a crash when accessed.

What breaks in production: Any malformed or incomplete request causes a server error instead of a proper client error response. Users won't understand what went wrong.

- **Duplicate SKU risk:** The system requires SKUs to be unique, but this code doesn't check before inserting. If a duplicate SKU is sent, the database may throw an error, but it's not handled.

What breaks in production: Duplicate SKUs may get created, or the system throws a raw database error instead of a clear message.

- **Two-step commit:** The product is committed first, and then inventory is created in a second transaction. If the second step fails, the product exists without inventory.

What breaks in production: Orphaned product records that are not linked to any warehouse.

- **Price is unchecked:** The code does not check if the price is valid, positive, or numeric. Invalid values can be stored or cause errors later.

What breaks in production: Incorrect calculations, negative pricing issues, or runtime errors.

- **No auth or ownership check:** There is no verification that the user is allowed to create products for the given warehouse. What breaks in production: Unauthorized users could manipulate other companies' data.
- **Invalid starting quantity:** Missing or negative stock values can make inventory incorrect from the first write.
- **Weak API responses:** The endpoint does not return proper status codes or a predictable error structure for clients.

Reworked version

Updated code is in the Part1-review, new.py file

Why this version is safer

- **Single transaction:** The product row and inventory row are committed together, so the database stays consistent.
- **Validation up front:** Bad requests are rejected cleanly before any write happens.
- **Conflict handling:** The API returns a proper 409 when a duplicate SKU is attempted.
- **Clear HTTP behavior:** The endpoint now uses 201 for a successful create and structured JSON for errors.
- **Cleaner business flow:** Optional fields, warehouse existence, and starting quantity are handled explicitly instead of implicitly.